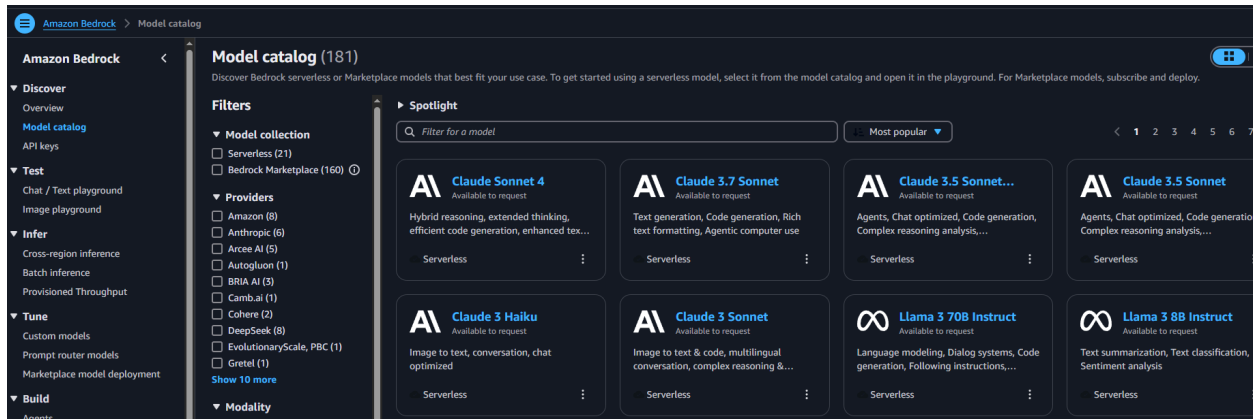


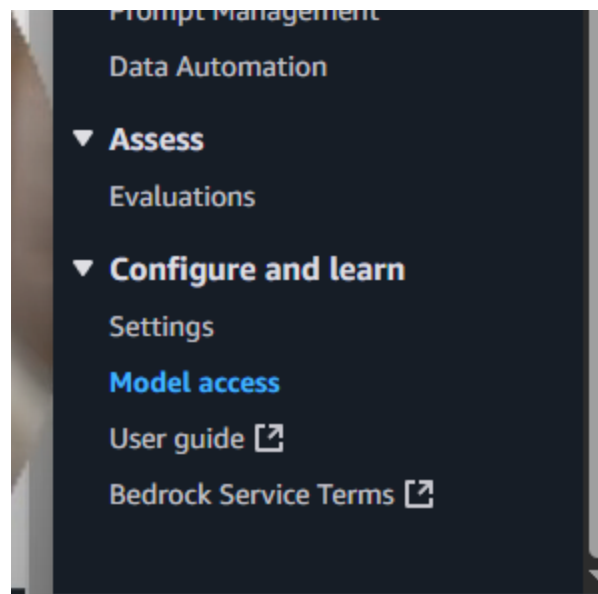
Complete Gen AI Project On AWS

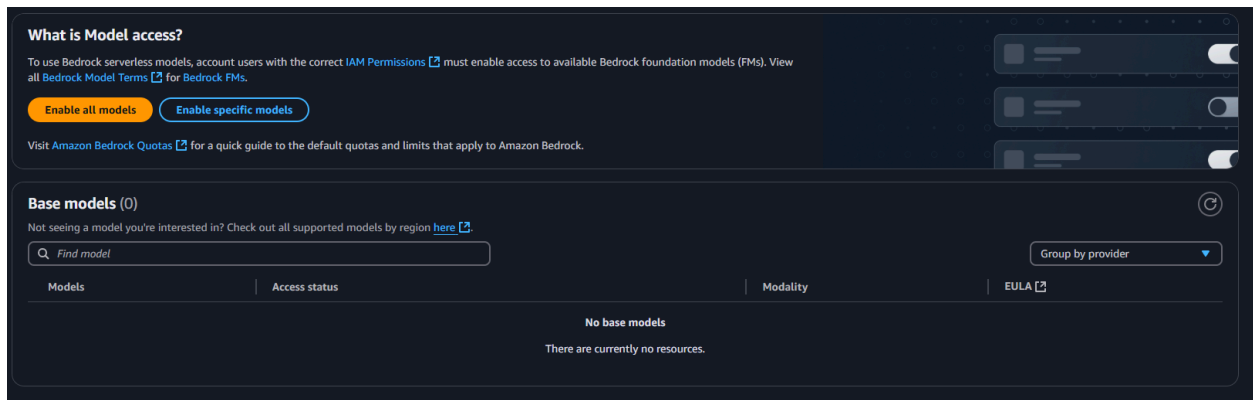
Bedrock



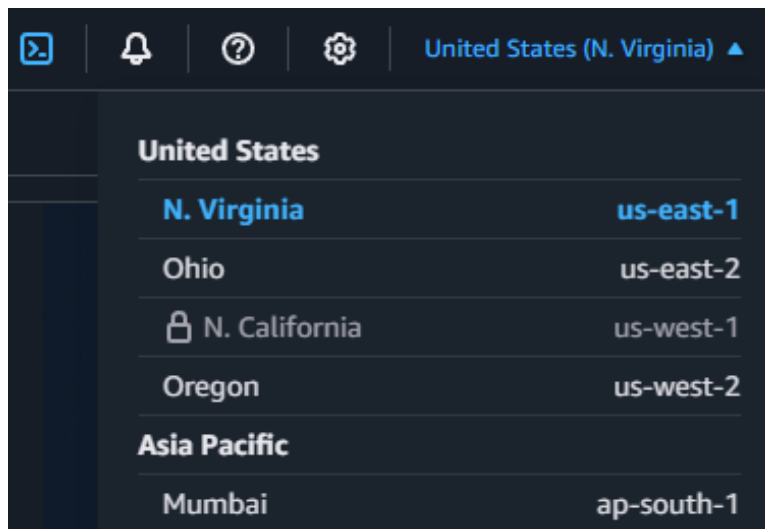
- These all are foundational models

Get Model Access





- Select region → US



- You'll see a model list

Base models (55) ⌵ Collapse all

Not seeing a model you're interested in? Check out all supported models by region [here](#)

Group by provider

Models	Access status	Modality	EULA
▼ AI21 Labs (3)	0/3 access granted		
Jamba-Instruct	🔒 Available to request	Text	EULA
Jamba 1.5 Large	🔒 Available to request	Text	EULA
Jamba 1.5 Mini	🔒 Available to request	Text	EULA
▼ Amazon (15)	0/15 access granted		
Titan Text G1 - Lite	🔒 Available to request	Text	EULA
Titan Text G1 - Express	🔒 Available to request	Text	EULA
Titan Image Generator G1	🔒 Available to request	Image	EULA
Titan Text G1 - Premier	🔒 Available to request	Text	EULA
Titan Multimodal Embeddings G1	🔒 Available to request	Embedding	EULA
Titan Embeddings G1 - Text	🔒 Available to request	Embedding	EULA
Titan Text Embeddings V2	🔒 Available to request	Embedding	EULA
Titan Image Generator G1 v2	🔒 Available to request	Image	EULA
Nova Sonic	🔒 Available to request	Speech	EULA
Nova Canvas	🔒 Available to request	Image	EULA

What is Model access?

To use Bedrock serverless models, account users with the correct **IAM Permissions** must accept **all Bedrock Model Terms** [for Bedrock FMs](#).

Modify model access

- Select the models and click next

Model access updates submitted ×

It may take several minutes to receive or remove access to models. Meanwhile, you can use other Bedrock console pages. Once your access is updated, you can use the models in Bedrock. Refresh the base models table to view the updated statuses.

AWS Lambda

Lambda is AWS's "serverless function" service.

It lets you run your code (Python, etc.) without managing any servers.

🧠 "Just give AWS a small function. They'll run it automatically whenever it's needed — and only charge you for those milliseconds."

📦 Example Use Cases

Scenario	How Lambda Helps
🧠 Auto-summarize a document uploaded to S3	Lambda gets triggered on upload, calls Bedrock to summarize
🖼️ Generate image using Bedrock	Lambda listens for user input, calls image model
🔄 Preprocess incoming data	Clean or filter text before storing in DB or S3
🔌 Expose ML models as API	With Lambda + API Gateway
📊 Trigger ETL or pipeline tasks	When a file is uploaded, or daily cron job runs

🔑 Key Features

Feature	Why It Matters for Gen AI
No servers	Zero infrastructure management (AWS handles scaling).
Pay-per-use	\$0.20 per 1M requests (cheap for low traffic).
Fast cold starts	~100ms latency (good for text/small-image AI).

🔄 What Triggers a Lambda?

You can configure it to run when:

Trigger Source	Example
S3	File upload (e.g., PDF to summarize)
API Gateway	User hits an endpoint (e.g., <code>/summarize</code>)
EventBridge	Run daily at 7PM
DynamoDB	A new record is added
SNS/SQS	New message arrives
Bedrock (via API call)	Trigger LLM model

Trigger Source	Example
Manual	You test it in AWS Console or SDK

How Does Lambda Work?

You upload your code → AWS stores it → You define a trigger → AWS runs the function when triggered

Gen AI Use Cases

1. Text Processing

- Summarize articles with Hugging Face models (e.g., `distilbart`).

```
import transformers
def lambda_handler(event, context):
    summarizer = transformers.pipeline("summarization")
    return summarizer(event["text"])
```

Cost: ~₹16/month for 10K summaries.

2. Image Metadata Tagging

- Use Lambda + Rekognition to auto-tag uploaded images.

3. Chatbots

- Connect Lambda to API Gateway for a ChatGPT-like API.

Limitations

- 15-minute max runtime** (not for heavy training).
- 10GB RAM limit** (use SageMaker/EC2 for big models).
- Cold starts** (first request may be slow).

Lambda + Bedrock Example (LLM Call)

```

import boto3, json

client = boto3.client("bedrock-runtime", region_name="us-east-1")

def lambda_handler(event, context):
    user_input = event['prompt']

    response = client.invoke_model(
        modelId="anthropic.claude-3-sonnet-20240229-v1:0",
        body=json.dumps({
            "prompt": f"Human: {user_input}\nAssistant:",
            "max_tokens_to_sample": 100
        }),
        contentType="application/json",
        accept="application/json"
    )

    result = json.loads(response['body'].read())
    return result

```

What This Code Does (High Level):

This code:

- Takes some text from a user (like a question or instruction)
- Sends it to an LLM (like Claude 3 Sonnet) via Amazon Bedrock
- Receives the model's response
- Returns it to the caller (e.g. a frontend or API)

import boto3, json

- `boto3` is the **official Python SDK for AWS** — it allows Python to talk to AWS services (like S3, EC2, Bedrock, etc.)
- `json` is a built-in Python module for converting data to/from JSON format (used when sending and receiving from Bedrock)

◆ `client = boto3.client("bedrock-runtime", region_name="us-east-1")`

This creates a **connection to Amazon Bedrock** service.

- `"bedrock-runtime"` : This means we want to **use the model (inference)**, not manage or list it
- `region_name="us-east-1"` : Bedrock only works in certain regions. `us-east-1` is currently one of the main ones.

Think of `client` as your remote control to talk to Bedrock from Python.

◆ `def lambda_handler(event, context):`

This is the **entry point** of the Lambda function — AWS looks for this function by name.

- `event` : The input data passed into the Lambda (e.g., a dictionary like `{ "prompt": "Tell me a joke" }`)
- `context` : AWS metadata (not needed for now)

🧠 For now: just know AWS calls this function when something triggers Lambda.

◆ `user_input = event['prompt']`

- This takes the `"prompt"` value from the input data
- If user sent `{ "prompt": "Who is the Prime Minister of India?" }`, then:

```
user_input = "Who is the Prime Minister of India?"
```

⚠️ If this key doesn't exist, it will crash. (We'll fix that later.)

Calling the Model via Bedrock

```
response = client.invoke_model(  
    modelId="anthropic.claude-3-sonnet-20240229-v1:0",  
    body=json.dumps(  
        "prompt": f"Human: {user_input}\nAssistant:",  
        "max_tokens_to_sample": 100  
    )  
)
```

```
contentType="application/json",  
accept="application/json"  
)
```

✅ **invoke_model()** : This method **sends your input to the chosen model** and gets the output.

json.dumps() → 🟢 Converts **Python dictionary** → **JSON string**

✅ **Parameters Explained:**

Parameter	Meaning
modelId	Which model to use. This is Claude 3 Sonnet's full ID
body	The actual prompt and settings, converted into JSON
prompt	What you want the model to answer
max_tokens_to_sample	Max words in the output (100 tokens ≈ 75 words)
contentType	You're sending JSON
accept	You want JSON response back

🧠 **The model expects prompts like this:**

Human: Tell me a joke.
Assistant:

◆ **result = json.loads(response['body'].read())**

This line:

1. Reads the model's reply (which is like a file-like object)
2. Converts it from JSON format to → **Python dictionary**

Example output:


```
{
  "completion": "Sure! Here's a joke: Why don't scientists trust atoms? Because they make up everything!"
}
```

Now

`result` is a Python dictionary you can use.

◆ **return result**

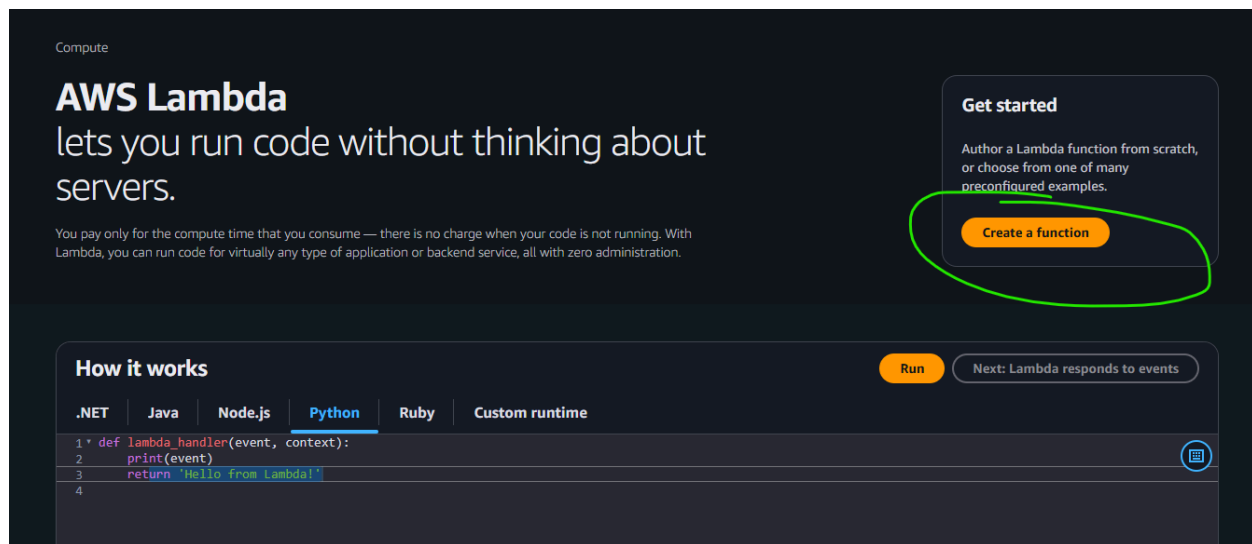
The final line **returns the model's answer** back to whoever triggered this Lambda (like a web app, S3 trigger, or API Gateway).

✗ Beginner Mistakes to Avoid

Mistake	Fix
✗ Writing long-running code (e.g. training a model)	✗ Lambda is NOT for training — use SageMaker or EC2
✗ Using too much memory/time	Max 15 minutes, 10 GB RAM
✗ Forgetting to handle <code>event</code> input correctly	✓ Always check keys exist (<code>event.get()</code>)
✗ No logs	✓ Use <code>print()</code> or <code>CloudWatch</code> to debug
✗ Not setting permissions	✓ Add Bedrock, S3, or other required IAM roles

Create Lambda Function

- Go to **Lambda → Create Function**



- Write function **name** & select Runtime as **Python**

The image shows the "Create function" form in the AWS console. At the top, it says "Create function" with an "Info" link. Below this, it says "Choose one of the following options to create your function." There are three radio button options: "Author from scratch" (selected), "Use a blueprint", and "Container image". The "Author from scratch" option has a subtext "Start with a simple Hello World example." Below the options is the "Basic information" section. It has three sub-sections: "Function name" with a text input field containing "AWSAppBedrock" and a note about naming rules; "Runtime" with a dropdown menu set to "Python 3.13" and a note about supported languages; and "Architecture" with two radio button options, "arm64" and "x86_64" (selected). At the bottom is the "Permissions" section with a note about the default execution role.

- Rest of it as it is
- Create Function

Getting started

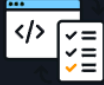
Now that you created your function, choose a path to continue developing.

Set up local development



Configure AWS CLI credentials

To enable secure access to AWS resources locally, create an access key in the IAM console and configure it with the AWS CLI for local development. [Learn more](#)



Code and test

Download your function code, edit locally, install dependencies, and test changes on your computer.



Deploy and invoke

Package your local changes into a ZIP file, deploy back to AWS Lambda in one step, and see it work in the cloud.

Connect to other services

Let other AWS services automatically run your function when events happen. [Add trigger](#)

Write a test

Send test data to your function and see how it responds. [Add test](#)

☐ Don't show again Dismiss

AWSAppBedrock

Throttle Copy ARN Actions

Function overview Info

Diagram Template

**AWSAppBedrock**

 Layers (0)

+ Add trigger

+ Add destination

Export to Infrastructure Composer

Download

Description

-

Last modified

1 minute ago

Function ARN

 arn:aws:lambda:ap-south-1:537976613696:function:AWSAppBedrock

Function URL

Info

- Edit the **timeout to 3 min**

Code Test Monitor **Configuration** Aliases Versions

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Tags

General configuration Info

Description

-

Memory

128 MB

Ephemeral storage

512 MB

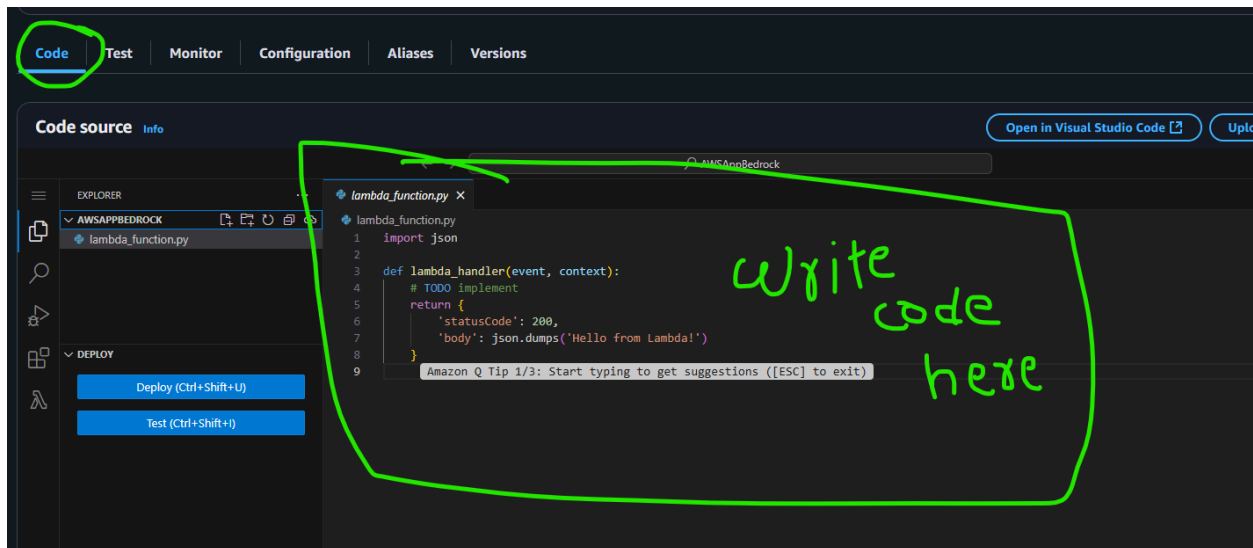
Timeout

0 min 3 sec

SnapStart

None

Edit



- Write in VS Code and copy paste in AWS later

`pip install boto3`

- `boto3` is used to invoke the foundation model

Requirement : Create a blog

- **Model:** *meta.llama3-8b-instruct-v1:0*

For Code, Refer → <https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-meta.html>

Standard prompt for llama3 models:

```
# Embed the prompt in Llama 3's instruction format.
formatted_prompt = f"""
<|begin_of_text|><|start_header_id|>user<|end_header_id|>
{prompt}
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
"""
```

```
formatted_prompt = f"""
<|begin_of_text|><|start_header_id|>user<|end_header_id|>
{prompt}
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
"""
```

- **User** is giving the query & **assistant** will generate a response

✓ Correct Way to Format Body in Bedrock Request

```
body = json.dumps({
    "prompt": f"<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n"
    f"{prompt_text}\n<|start_header_id|>assistant<|end_header_id|>\n",
    "max_gen_len": 400,
    "temperature": 0.7,
    "top_p": 0.9
})
```

```
def blog_generate_using_bedrock(topic: str) → str:
    prompt = f"""
    <|begin_of_text|><|start_header_id|>user<|end_header_id|>
    Write a 200 word blog on the topic: {topic}
    <|eot_id|>
    """
```

```
<|start_header_id|>assistant<|end_header_id|>
"""
```

```
body = {
    "prompt" : prompt,
    "max_gen_len": 400,
    "temperature": 0.7,
    "topP": 0.9
}
```

Call the model:

```
client = boto3.client("bedrock-runtime", region_name="us-east-1")
modelId = "meta.llama3-8b-instruct-v1:0"

try:
    response = client.invoke_model(modelId= modelId, body= json.dumps(body))

except (ClientError, Exception) as e:

    print(f"ERROR: Can't invoke '{modelId}'. Reason: {e}")
    exit(1)
```

```
# Decode the response body.
model_response = json.loads(response.get('body').read()) # or you can also write response["body"].read()

# Extract and print the response text.
blog = model_response["generation"]
```

```
print ("model_response: ", model_response)
print ("Blog: ", blog)
```

Capture the blog topic:

- We have to incorporate this into `lambda_handler` function

```
def lambda_handler(event, context):
    event = json.loads(event['body'])
    blogtopic = event['blogtopic']

    blog_generate_using_bedrock(blogtopic=blogtopic)
```

- `event` : contains the **input data** sent to Lambda (e.g. from API Gateway, HTTP call, or other service)
- `context` : contains **metadata** about the invocation (not used here, but useful for logging, timeouts, etc.)

What's Inside `event['body']` ?



The `body` comes from the **API Gateway HTTP request**.

The `body` comes from the **API Gateway HTTP request**. When you send a `POST` request from Postman to your API Gateway endpoint, the data you put in the "Body" section of your Postman request is what API Gateway wraps up and sends to your Lambda function inside the `event` dictionary, specifically under a key named `'body'`.

In AWS Lambda, the `event` is the **data sent to the function when it's invoked**.

When Lambda is triggered through **API Gateway**, the data structure looks like this:

Example `event` for API Gateway (HTTP POST):

If your Lambda is triggered from an **API Gateway**, then the `event` looks like this:

```
{
  "resource": "/",
  "path": "/",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "body": "{\"blogtopic\": \"How AI is changing education\"}",
  "isBase64Encoded": false
}
```

So:

- `event['body']` is a **string** that looks like JSON
- We use `json.loads()` to **convert** that string → into a real **Python dictionary**

Example:

```
event = json.loads('{"blogtopic": "Importance of Sleep"}')
```

➡ After this, `event` becomes:

```
{ "blogtopic": "Importance of Sleep" }
```

✅ Now you can work with it like a normal Python dict.

 `blogtopic = event['blogtopic']`

This line:

- Extracts the value of `"blogtopic"` from the parsed `event` dictionary

Example:

```
blogtopic = "Importance of Sleep"
```

So now the topic for the blog is saved in a variable named `blogtopic`.

Tried this in VS Code:

```
import boto3
import botocore.config
import json
from botocore.exceptions import ClientError

def blog_generate_using_bedrock(topic: str) → str:
    prompt = f"""
    <|begin_of_text|><|start_header_id|>user<|end_header_id|>
    Write a 200 word blog on the topic: {topic}
    <|eot_id|>
    <|start_header_id|>assistant<|end_header_id|>
    """
    body = {
        "prompt": prompt,
        "max_gen_len": 400,
        "temperature": 0.7,
        "top_p": 0.9
    }

    #Call the model

    client = boto3.client("bedrock-runtime", region_name="us-east-1")
    modelId = "meta.llama3-8b-instruct-v1:0"

    try:
```

```

        response = client.invoke_model(modelId= modelId, body= json.dumps(body))

    except (ClientError, Exception) as e:

        print(f"ERROR: Can't invoke '{modelId}'. Reason: {e}")
        exit(1)

    # Decode the response body.
    model_response = json.loads(response.get('body').read()) # or you can also write response["body"].read()

    # Extract and print the response text.
    blog = model_response["generation"]

    print ("model_response: ", model_response)
    print ("Blog: ", blog)

def lambda_handler(event, context):
    event = json.loads(event['body'])
    blogtopic = event['blogtopic']

    blog_content = blog_generate_using_bedrock(topic=blogtopic)

    return {
        "statusCode": 200,
        "body": json.dumps({"blog": blog_content})
    }
blog_generate_using_bedrock("cricket")

```

Output:

```

model_response: {'generation': ' **The Gentleman\'s Game: Why Cricket Remains a Beloved Sport**\n\nCricket, often referred to as the "gentleman\'s game," has been a beloved sport for centuries. Originating in England in the 16th c

```

entury, cricket has evolved into a global phenomenon, with millions of fans worldwide. The game's unique blend of strategy, skill, and sportsmanship has captivated audiences, making it a staple of international sporting events.

One of the key reasons cricket's popularity endures is its rich history and tradition. The game has been played by British aristocrats and working-class folk alike, with the first recorded match dating back to 1598. The sport's aristocratic roots are still reflected in its formal attire, with players donning whites and caps, evoking a sense of elegance and refinement.

Cricket's global appeal is also due to its adaptability. The game has been played in various forms, from the fast-paced, high-scoring Twenty20 format to the more strategic, longer forms of the game. This diversity has allowed cricket to transcend cultural and linguistic barriers, making it a beloved sport in countries like India, Australia, Pakistan, and the West Indies.

In addition to its global reach, cricket's sporting merit is undeniable. The game requires a unique combination of physical and mental skills, including hand-eye coordination, strategy, and mental toughness. The thrill of watching a well-executed shot or a masterful spell of bowling is unmatched, making cricket a true test of skill and character.

As the sport continues to evolve, it's clear that cricket's enduring popularity is a testament to its timeless appeal. Whether you're a seasoned fan or a newcomer to the game, cricket's unique blend of history, tradition, and sporting excellence ensures that it will remain a beloved sport for generations to come.'

```
'prompt_token_count': 29, 'generation_token_count': 369, 'stop_reason': 'stop'}
```

Blog: ****The Gentleman's Game: Why Cricket Remains a Beloved Sport****

Cricket, often referred to as the "gentleman's game," has been a beloved sport for centuries. Originating in England in the 16th century, cricket has evolved into a global phenomenon, with millions of fans worldwide. The game's unique blend of strategy, skill, and sportsmanship has captivated audiences, making it a staple of international sporting events.

One of the key reasons cricket's popularity endures is its rich history and tradition. The game has been played by British aristocrats and working-class folk alike, with the first recorded match dating back to 1598. The sport's aristocratic

roots are still reflected in its formal attire, with players donning whites and caps, evoking a sense of elegance and refinement.

Cricket's global appeal is also due to its adaptability. The game has been played in various forms, from the fast-paced, high-scoring Twenty20 format to the more strategic, longer forms of the game. This diversity has allowed cricket to transcend cultural and linguistic barriers, making it a beloved sport in countries like India, Australia, Pakistan, and the West Indies.

In addition to its global reach, cricket's sporting merit is undeniable. The game requires a unique combination of physical and mental skills, including hand-eye coordination, strategy, and mental toughness. The thrill of watching a well-executed shot or a masterful spell of bowling is unmatched, making cricket a true test of skill and character.

As the sport continues to evolve, it's clear that cricket's enduring popularity is a testament to its timeless appeal. Whether you're a seasoned fan or a newcomer to the game, cricket's unique blend of history, tradition, and sporting excellence ensures that it will remain a beloved sport for generations to come.

✨ Further modifications

- We'll save the response in an **S3 Bucket** in `.txt` format

```
from datetime import datetime
```

```
if blog_content:
```

```
    current_time = datetime.now().strftime('%H%M%S')
```

```
    s3_key = f"blog-output-folder/{current_time}.txt"
```

```
    s3_bucket = "awsblog983" #make sure the name is unique for the region
```

```
    save_blog_details_s3(s3_key, s3_bucket, blog_content) #Created the function below
```

```
else:  
    print("No blog was generated.")
```

`s3_key` → File name in the folder `blog-output-folder`

- `Current_time.txt`

Create another function to save the blog in S3

```
def save_blog_details_s3(s3_key, s3_bucket, blog_content):  
    s3 = boto3.client('s3')  
  
    try:  
        s3.put_object(Bucket = s3_bucket, Key = s3_key, Body =blog_content)  
        print("Blog saved to s3")  
  
    except Exception as e:  
        print("Error when saving the blog to s3")
```

- `boto3.client('s3')` creates a connection to Amazon S3.
- `s3.put_object(Bucket = s3_bucket, Key = s3_key, Body = blog_content)` : This line **uploads the blog to S3**.

Parameter	What it means
<code>Bucket = s3_bucket</code>	The S3 bucket name (like the folder or drive you're uploading to)
<code>Key = s3_key</code>	The file path or name in S3 (e.g., <code>"blogs/cricket.txt"</code>)
<code>Body = blog_content</code>	The actual text of the blog that you're uploading

- We defined the above things in 📌

```

if blog_content:
    current_time = datetime.now().strftime('%H%M%S')
    s3_key = f"blog-output-folder/{current_time}.txt"
    s3_bucket = "aws_blog"

```

Entire code:

```

import boto3
import botocore.config
import json
from botocore.exceptions import ClientError
from datetime import datetime

def blog_generate_using_bedrock(topic: str) → str:
    prompt = f"""
    <|begin_of_text|><|start_header_id|>user<|end_header_id|>
    Write a 200 word blog on the topic: {topic}
    <|eot_id|>
    <|start_header_id|>assistant<|end_header_id|>
    """
    body = {
        "prompt": prompt,
        "max_gen_len": 300,
        "temperature": 0.7,
        "top_p": 0.9
    }

    #Call the model

    client = boto3.client("bedrock-runtime", region_name="us-east-1")
    modelId = "meta.llama3-8b-instruct-v1:0"

    try:

```

```

        response = client.invoke_model(modelId= modelId, body= json.dumps(body))

    except (ClientError, Exception) as e:

        print(f"ERROR: Can't invoke '{modelId}'. Reason: {e}")
        exit(1)

    # Decode the response body.
    model_response = json.loads(response.get('body').read()) # or you can also write response["body"].read()

    # Extract and print the response text.
    blog = model_response["generation"]

    return blog

def save_blog_details_s3(s3_key, s3_bucket, blog_content):
    s3 = boto3.client('s3')

    try:
        s3.put_object(Bucket = s3_bucket, Key = s3_key, Body =blog_content)
        print("Blog saved to s3")

    except Exception as e:
        print("Error when saving the blog to s3")

def lambda_handler(event, context):
    event = json.loads(event['body'])
    blogtopic = event['blogtopic']

    blog_content = blog_generate_using_bedrock(topic=blogtopic)

    if blog_content:

```

```
current_time = datetime.now().strftime('%H%M%S')
s3_key = f"blog-output-folder/{current_time}.txt"
s3_bucket = "aws_blog"
save_blog_details_s3(s3_key, s3_bucket, blog_content)
```

else:

```
print("No blog was generated.")
```

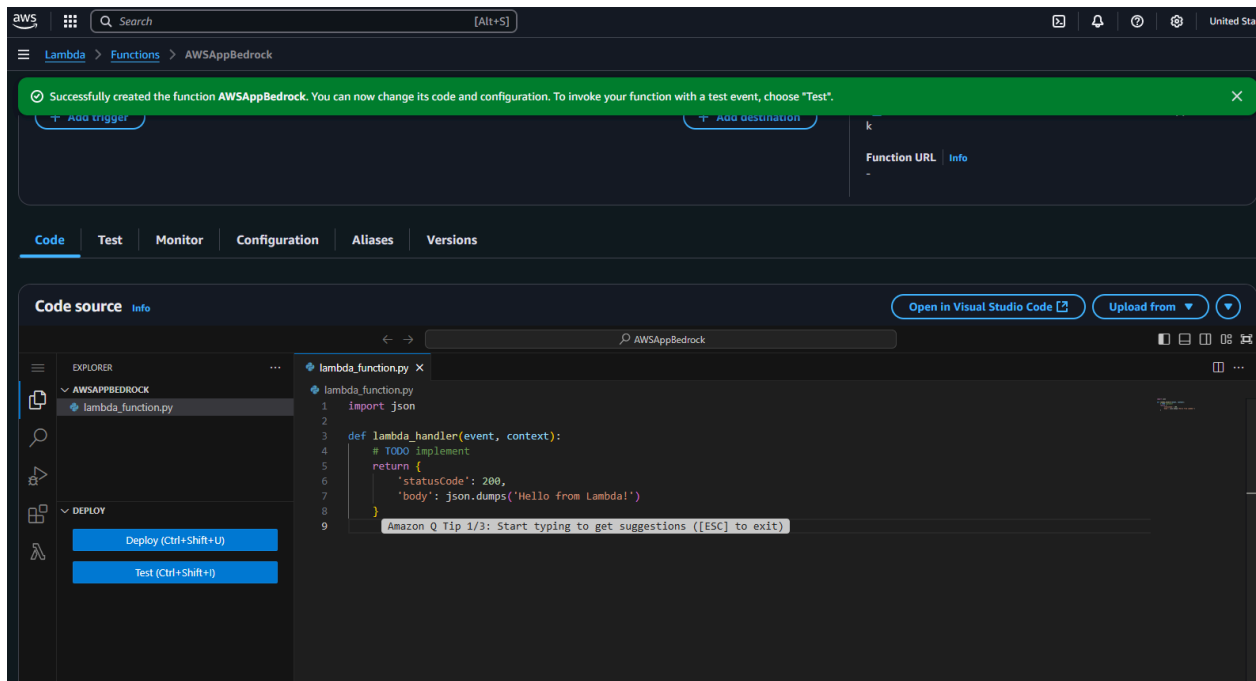
return {

```
    "statusCode": 200,
```

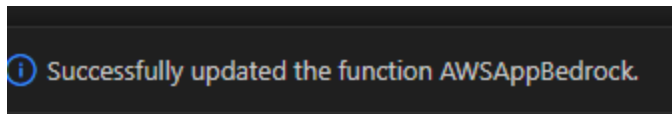
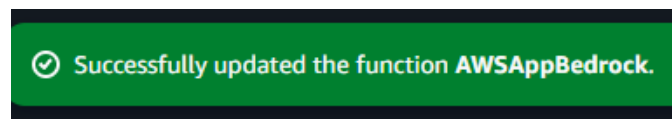
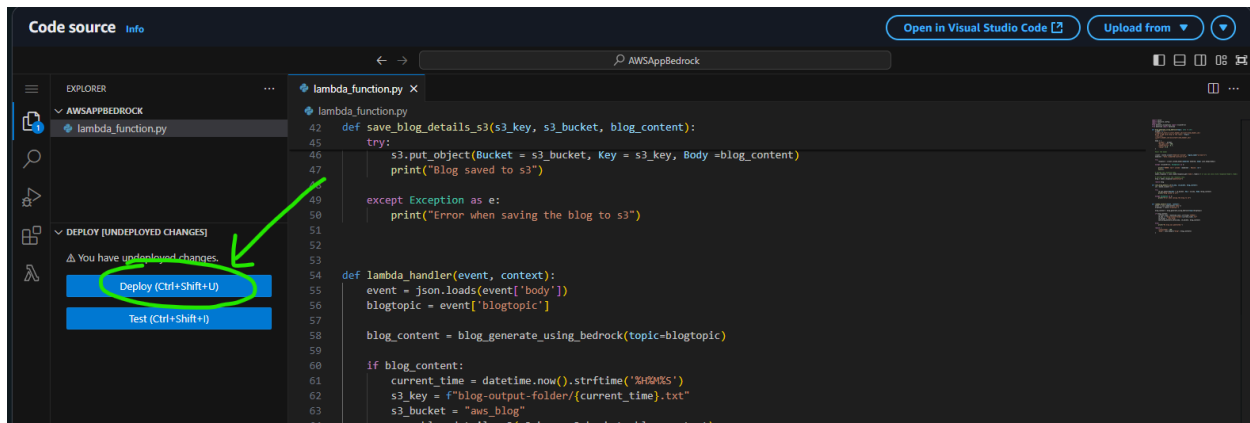
```
    "body": json.dumps({"blog": blog_content})
```

```
}
```

- Paste this in AWS Lambda



- Click **Deploy**

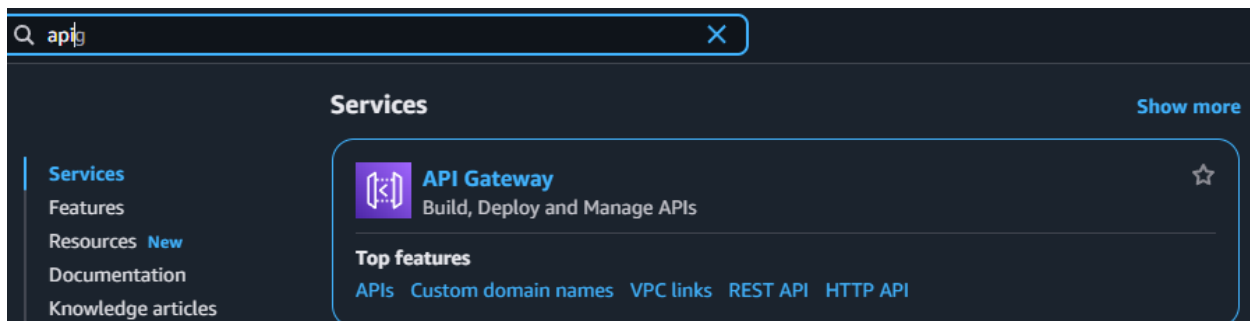


Create API Gateway

- You need API Gateway (or a **Lambda Function URL**) to turn your Lambda into a public HTTP endpoint that Postman or your frontend can call. API Gateway is the “front door” that accepts HTTP requests and hands them to your Lambda.
- **You need API Gateway** for *your specific goal* of triggering this code from Postman (or a web browser, or another application outside of AWS). You don't *always* need API Gateway to run a Lambda function, but you need it to expose your function via a web address (HTTP endpoint).

✓ Do You *Have* to Use API Gateway?

- **If you want an HTTP(S) URL** that anyone (or your frontend) can call → **Yes**, you need API Gateway (or the newer Lambda Function URL feature).
- If you only run your code manually in AWS Console, or invoke it from another AWS service (S3 trigger, CloudWatch Events, another Lambda), you **don't** need API Gateway.



- Create an API

Networking & Content Delivery

API Gateway

Create and manage APIs at scale

Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs.

Get started

Create a new API to begin exploring API Gateway. You can also import an external definition file into API Gateway.

[Create an API](#)

How it works

API Gateway enables you to connect and access data, business logic, and functionality from backend services such as workloads running on Amazon Elastic Compute Cloud (Amazon EC2), code running on AWS Lambda, any web application, or real-time communication applications.

Pricing

With Amazon API Gateway, you only pay when your APIs are in use. There are no minimum fees or upfront commitments. For HTTP and REST APIs, you pay based on API calls received and amount of data transferred out. For WebSocket APIs, you pay based on number of messages and connection duration.

[Learn more about pricing](#)

Resources

- [Getting Started Guide](#)
- [Developer Guide](#)
- [API References](#)

- We'll be using HTTP API

HTTP API

Build low-latency and cost-effective REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.

Works with the following:
Lambda, HTTP backends

Import

Build

WebSocket API

Build a WebSocket API using persistent connections for real-time use cases such as chat applications or dashboards.

Works with the following:
Lambda, HTTP, AWS Services

Build

Configure API

API details

API name

An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

bedrockdemoapi

IP address type | Info

Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

☒ IPv4

Includes only IPv4 addresses.

☐ Dualstack

Includes IPv4 and IPv6 addresses.

Integrations (0) | Info

Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

Add integration

Cancel

Review and create

Next

Add Routes:

Create a route

Route and method | Info

Route name eg. /pets

Choose a method and enter a path to create a route. You can also specify one \$default route per API. The \$default route is invoked when the request to the API matches no other routes.

POST

/blog-generation

Cancel

Create

Attach Integration:

Complete Gen AI Project On AWS

27

Routes

Routes for bedrockdemoapi

▼ /blog-generation

POST

Create

Route details

POST /blog-generation (ID: lbzjdhg)

ARN
arn:aws:apigateway:us-east-1::apis/3f971d9l0l/routes/lbzjdhg

Authorization
Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route. [Attach authorization](#)

Integration
The integration is the backend resource that this route calls when it receives a request.

No integration attached to this route. [Attach integration](#)

- Attach to lambda function

Create an integration

Attach this integration to a route

×

Integration target

Integration type

Lambda function ▼

Integration details

Integration target
Choose the Lambda function that API Gateway invokes when the route receives a request.

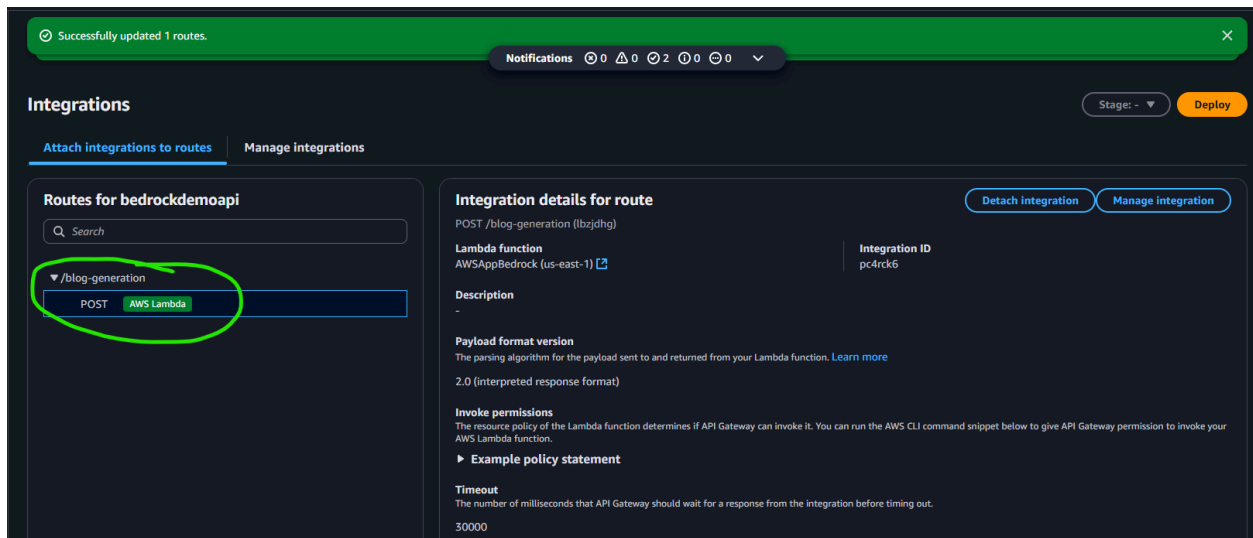
AWS Region
us-east-1 ▼

Lambda function
 ×

Name

► Advanced settings

Description - optional

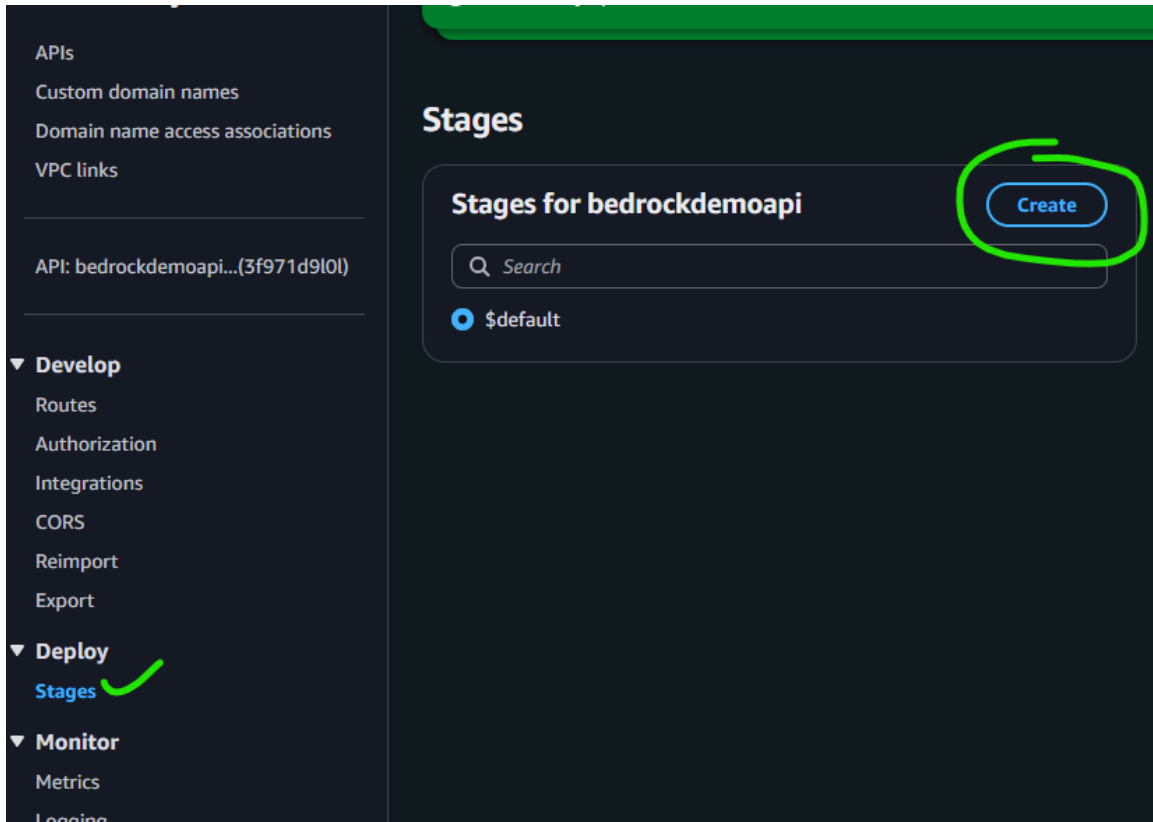


! Create a stage

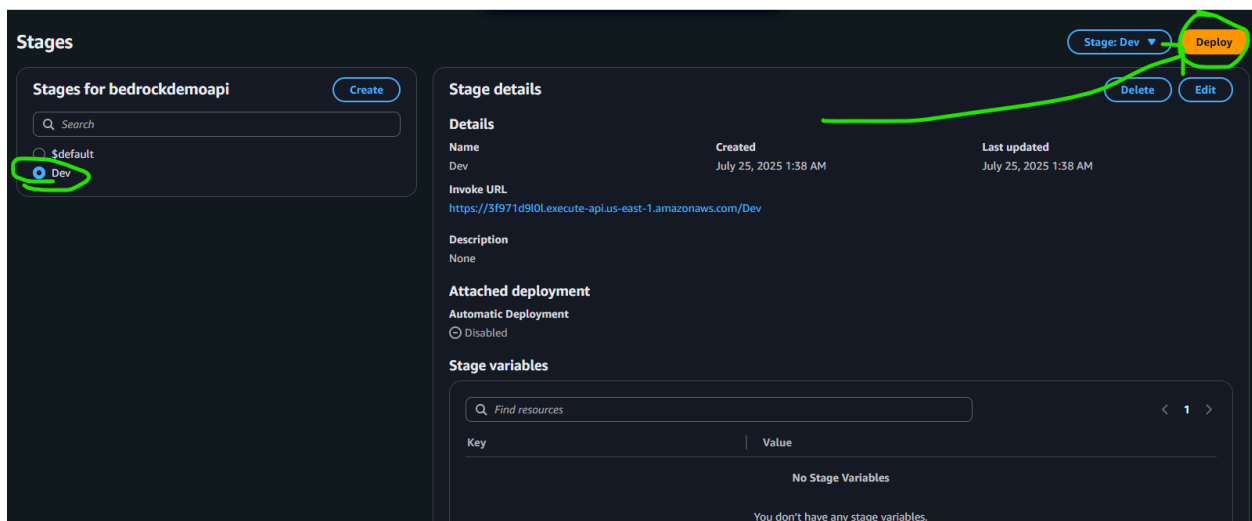
After you create your API, define its resources and methods (like `POST /blog-generation`), and integrate it with your Lambda function, you then **deploy it to a "stage"**.

Why do we need to add `Dev` if there's already a `$default` option?

While there is indeed a `$default` stage, it's generally **not recommended to use the `$default` stage for active development or production deployments**. It's often there for historical reasons, or for very simple, single-stage deployments, but it comes with limitations and best practice considerations.



- Just add name & create
- Deploy



Deploy API

Select an existing stage or [create a new stage](#) where your API will be deployed. You can edit the stage details to change the active deployment or turn on auto-deploy for a stage. If you turn on auto-deploy, you can't manually deploy your stage.

⚠ When you deploy an API to an existing stage, you immediately overwrite the current stage configuration with a new active deployment.

Stage

Dev

Deployment description

Cancel

Deploy

Stage details

Delete Edit

Details

Name	Created	Last updated
Dev	July 25, 2025 1:38 AM	July 25, 2025 1:38 AM

Invoke URL

<https://3f971d9l0l.execute-api.us-east-1.amazonaws.com/Dev> API

Description

None

Attached deployment

Automatic Deployment

⊖ Disabled

Stage variables

Find resources

Create S3 Bucket

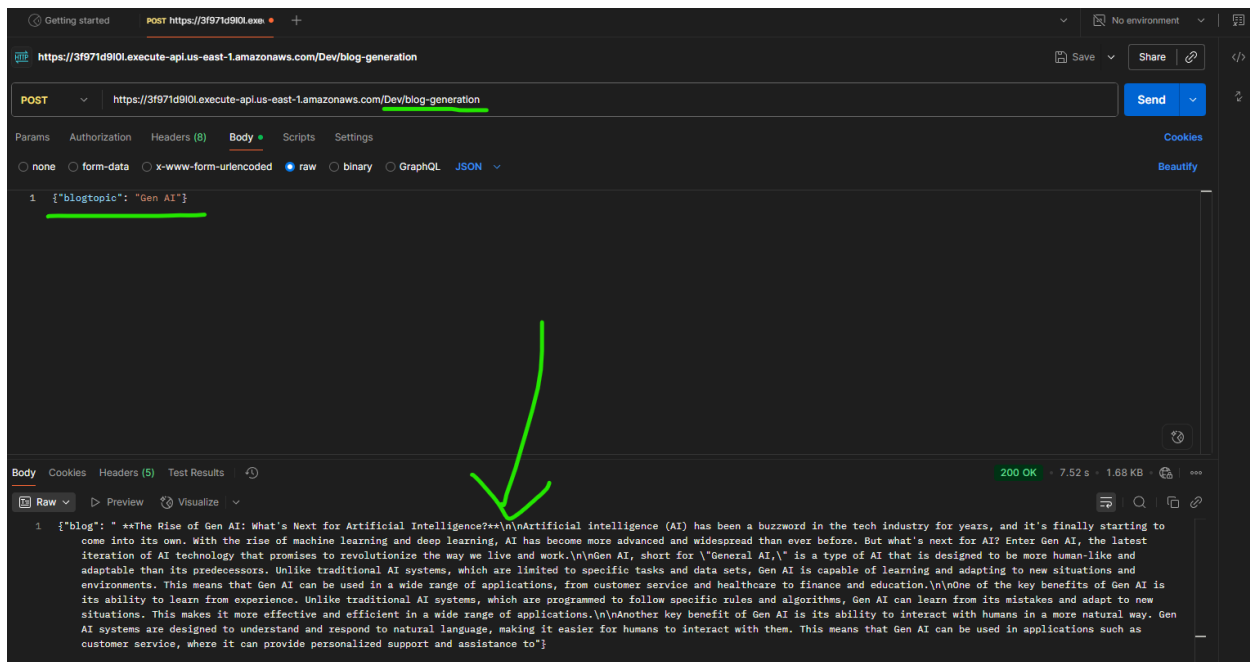
- Name= `awsblog983`
- Make sure the name is not taken by anyone else in the region

```
if blog_content:
    current_time = datetime.now().strftime('%H%M%S')
    s3_key = f"blog-output-folder/{current_time}.txt"
    s3_bucket = "awsblog983"
    save_blog_details_s3(s3_key, s3_bucket, blog_content)
```

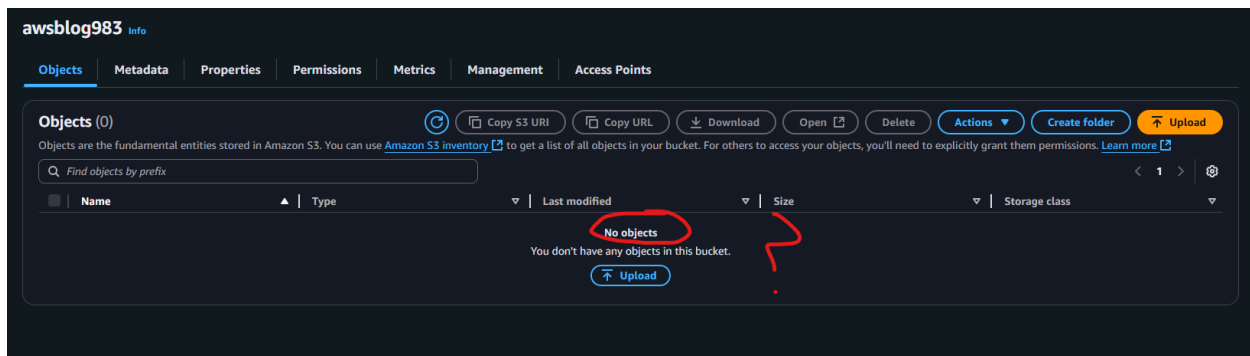
- Update the name in code & deploy

Test

- Install postman



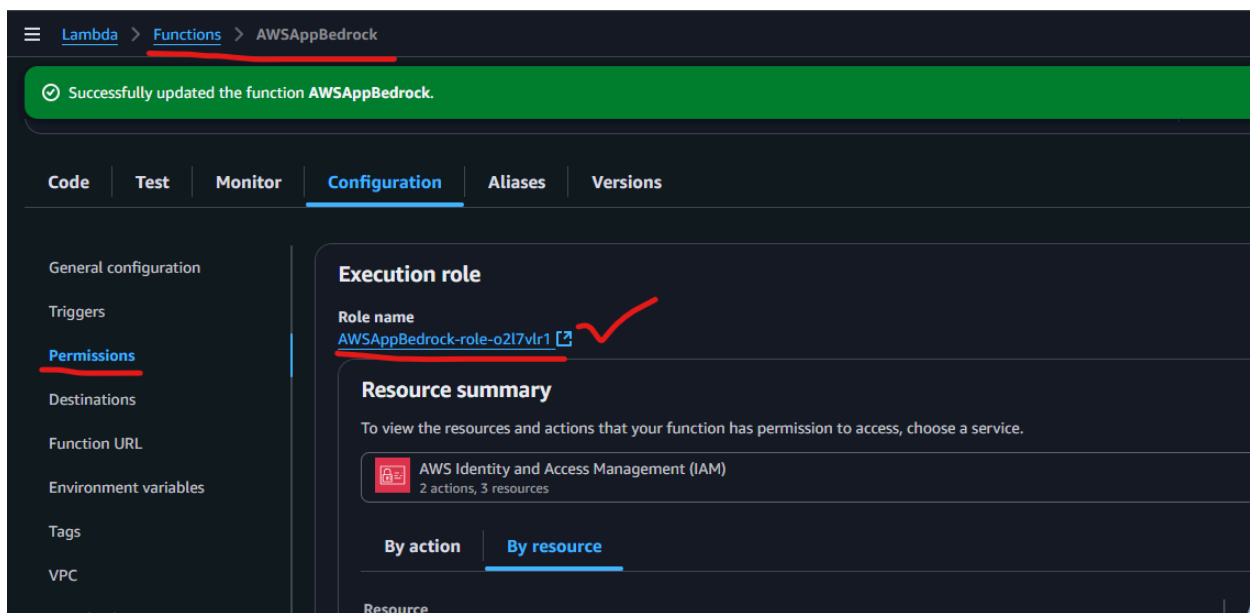
- Blog loaded but txt not added inside S3 bucket



Lambda Execution Role Permissions for S3 (Most Common Culprit)

The specific permission needed is

s3:PutObject



AWSAppBedrock-role-o2l7v1r1 [Info](#) [Delete](#) [Edit](#)

Summary

Creation date: July 25, 2025, 01:16 (UTC+05:30)

Last activity: 6 minutes ago

ARN: `arn:aws:iam::537976613696:role/service-role/AWSAppBedrock-role-o2l7v1r1`

Maximum session duration: 1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) [Info](#)

You can attach up to 10 managed policies.

[Simulate](#) [Remove](#) [Add permissions](#)

[Attach policies](#) [Create inline policy](#)

Filter by Type: All types

☐	Policy name	Type	Attached entities
☐	AmazonBedrockFullAccess	AWS managed	1
☐	AWSLambdaBasicExecutionRole-670800ec-7add-44c1-bd04-b5...	Customer managed	1

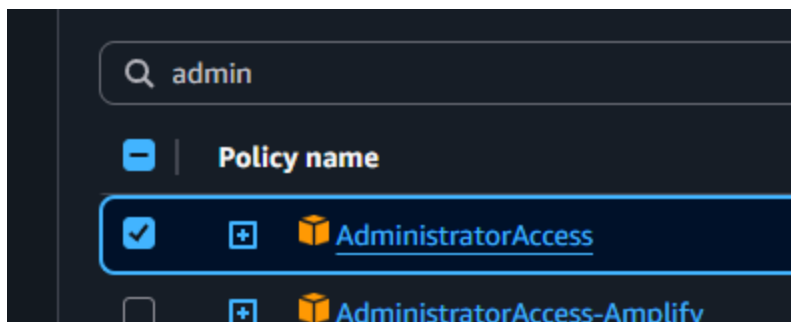
► **Permissions boundary** (not set)

▼ **Generate policy based on CloudTrail events**

You can generate a new policy based on the access activity for this role, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

[Generate policy](#)

- Provide admin access



- Run postman again

awsblog983 [Info](#)

Objects | Metadata | Properties | Permissions | Metrics | Management | Access Points

Objects (1) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	blog-output-folder/	Folder	-	-	-

- Now it's added in S3

Amazon S3 > Buckets > awsblog983 > blog-output-folder/

blog-output-folder/

Objects

Properties

Objects (1)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permission

Find objects by prefix

	Name	Type	Last modified	Size	Storage class
☐	 211432.txt	txt	July 25, 2025, 02:44:34 (UTC+05:30)	1.5 KB	Standard